

Corrigé – Examen d'entraînement LEPL1110

Éléments Finis

Équipe pédagogique — Mai 2026

1 Équation de diffusion de la chaleur en 1D

1.1 Formulation faible en espace

On multiplie $\partial_t u - \kappa u'' = 0$ par $v \in H^1(0, 1)$ et on intègre sur $(0, 1)$:

$$\int_0^1 \partial_t u v \, dx - \kappa \int_0^1 u'' v \, dx = 0.$$

On applique l'intégration par parties au second terme :

$$-\kappa \int_0^1 u'' v \, dx = \kappa \int_0^1 u' v' \, dx - \kappa [u' v]_0^1 = \kappa \int_0^1 u' v' \, dx - \underbrace{\kappa u'(1)}_{=0} v(1) + \underbrace{\kappa u'(0)}_{=0} v(0).$$

Les termes de bord s'annulent grâce aux conditions de Neumann homogènes. La formulation faible est donc :

$$\boxed{\int_0^1 \partial_t u v \, dx + \kappa \int_0^1 u' v' \, dx = 0, \quad \forall v \in H^1(0, 1).}$$

Pourquoi les conditions de Neumann n'apparaissent-elles pas ? Les conditions de Neumann homogènes sont des *conditions naturelles* : elles sont automatiquement satisfaites par la formulation faible, car le terme de bord issu de l'intégration par parties vaut $\kappa(u'(1)v(1) - u'(0)v(0)) = 0$ pour tout v . On ne doit pas les imposer sur l'espace d'approximation. En revanche, les conditions de Dirichlet (essentiels) doivent être imposées explicitement.

Erreur classique : confondre conditions essentielles (Dirichlet, à imposer sur l'espace de fonctions test) et conditions naturelles (Neumann, issues de la formulation faible). Ici il n'y a aucune condition essentielle : v est quelconque dans $H^1(0, 1)$.

1.2 Semi-discrétisation en espace

On substitue $u_h(x, t) = \sum_j u_j(t) \varphi_j(x)$ dans la formulation faible et on prend $v = \varphi_i$:

$$\sum_j \underbrace{\left(\int_0^1 \varphi_j \varphi_i \, dx \right)}_{M_{ij}} \dot{u}_j(t) + \kappa \sum_j \underbrace{\left(\int_0^1 \varphi_j' \varphi_i' \, dx \right)}_{K_{ij}} u_j(t) = 0, \quad \forall i.$$

En notation matricielle :

$$\boxed{M \dot{\mathbf{u}}(t) + \kappa K \mathbf{u}(t) = \mathbf{0}.$$

M est symétrique définie positive. K est symétrique semi-définie positive : son noyau contient les fonctions constantes (gradient nul $\Rightarrow K_{ij}$ nul pour $\varphi = 1$).

1.3 Matrices élémentaires et assemblage ($N = 2$)

(a) Matrices élémentaires

Fonctions de forme sur $[x_e, x_{e+1}]$: $\varphi'_0(x) = -1/h$ et $\varphi'_1(x) = +1/h$.

Matrice de rigidité :

$$K_{00}^{(e)} = \int_{x_e}^{x_{e+1}} \frac{1}{h^2} dx = \frac{1}{h}, \quad K_{01}^{(e)} = -\frac{1}{h}, \quad K_{11}^{(e)} = \frac{1}{h}.$$

$$K^{(e)} = \frac{1}{h} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}.$$

Matrice de masse : avec le changement de variable $\xi = (x - x_e)/h$:

$$M_{00}^{(e)} = h \int_0^1 (1 - \xi)^2 d\xi = \frac{h}{3}, \quad M_{01}^{(e)} = h \int_0^1 \xi(1 - \xi) d\xi = \frac{h}{6}, \quad M_{11}^{(e)} = \frac{h}{3}.$$

$$M^{(e)} = \frac{h}{6} \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

Erreur classique : oublier le facteur h dans $M^{(e)}$, ou omettre le changement de variable. On obtient alors une matrice de masse deux fois trop grande ou trop petite.

(b) Assemblage global pour $N = 2$, $h = 1/2$

Pour $h = 1/2$: $K^{(e)} = 2 \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$ et $M^{(e)} = \frac{1}{12} \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$.

Après assemblage des contributions aux nœuds 0, 1, 2 (le nœud 1 est partagé) :

$$K = \begin{pmatrix} 2 & -2 & 0 \\ -2 & 4 & -2 \\ 0 & -2 & 2 \end{pmatrix}, \quad M = \frac{1}{12} \begin{pmatrix} 2 & 1 & 0 \\ 1 & 4 & 1 \\ 0 & 1 & 2 \end{pmatrix}.$$

Détail pour K_{11} : le nœud $x_1 = 1/2$ appartient à l'élément (1) (nœud local 1) et à l'élément (2) (nœud local 0), donc $K_{11} = K_{11}^{(1)} + K_{00}^{(2)} = 2 + 2 = 4$.

(c) Vérification $K\mathbf{1} = \mathbf{0}$

$$K \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 - 2 + 0 \\ -2 + 4 - 2 \\ 0 - 2 + 2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}. \quad \checkmark$$

Interprétation physique : une température uniforme $u = \text{const}$ ne génère aucun flux de chaleur ($\partial_x u = 0$ partout). L'énergie bilinéaire $\kappa \int u'v' dx$ est nulle pour toute v , ce qui se traduit au niveau discret par $K\mathbf{1} = \mathbf{0}$. La température moyenne est conservée ($\mathbf{1}^\top M \dot{\mathbf{u}} = -\kappa \mathbf{1}^\top K \mathbf{u} = 0$).

Erreur classique : croire que K est définie positive. Elle n'est que semi-définie positive : son plus petit valeur propre est 0. Sans condition de Dirichlet, le système $K\mathbf{u} = \mathbf{f}$ n'a pas de solution unique (système singulier).

1.4 Schéma en temps et stabilité L^2

(a) Schéma θ

$$M \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} + \kappa K (\theta \mathbf{u}^{n+1} + (1 - \theta) \mathbf{u}^n) = \mathbf{0}.$$

En réarrangeant :

$$(M + \theta \kappa \Delta t K) \mathbf{u}^{n+1} = (M - (1 - \theta) \kappa \Delta t K) \mathbf{u}^n.$$

θ	Schéma
0	Euler explicite
1	Euler implicite
1/2	Crank–Nicolson

(b) Stabilité L^2

Problème continu. On multiplie $\partial_t u - \kappa u'' = 0$ par u et on intègre :

$$\frac{1}{2} \frac{d}{dt} \|u\|_{L^2}^2 = -\kappa \int_0^1 (u')^2 dx \leq 0.$$

La norme L^2 est donc décroissante.

Schéma discret. En multipliant le schéma θ par $\mathbf{u}^{n+\theta} = \theta \mathbf{u}^{n+1} + (1 - \theta) \mathbf{u}^n$ et en utilisant la symétrie de M , on obtient :

$$\frac{1}{\Delta t} (\|\mathbf{u}^{n+1}\|_M^2 - \|\mathbf{u}^n\|_M^2) = -2\left(\theta - \frac{1}{2}\right) \frac{\|\mathbf{u}^{n+1} - \mathbf{u}^n\|_M^2}{\Delta t} - 2\kappa \|\mathbf{u}^{n+\theta}\|_K^2.$$

Le second terme est toujours ≤ 0 . Le premier est ≤ 0 si et seulement si $\theta \geq 1/2$.

$$\text{Le schéma } \theta \text{ est inconditionnellement } L^2\text{-stable} \iff \theta \geq \frac{1}{2}.$$

Erreur classique : croire que Crank–Nicolson ($\theta = 1/2$) est marginalement instable. Il est stable, mais sans dissipation numérique supplémentaire : les modes ne sont pas amortis au-delà de l'amortissement physique.

(c) Condition de stabilité pour $\theta = 0$

En décomposant $\mathbf{u}^n$ sur les modes propres généralisés $K\phi = \lambda M\phi$, chaque amplitude a^n évolue comme $a^{n+1} = (1 - \kappa \Delta t \lambda) a^n$. La condition $|1 - \kappa \Delta t \lambda| \leq 1$ pour tout $\lambda \geq 0$ impose :

$$\Delta t \leq \frac{2}{\kappa \lambda_{\max}}.$$

1.5 Analyse modale et solution exacte

(a) Modes propres

On cherche $-\phi'' = \lambda\phi$ avec $\phi'(0) = \phi'(1) = 0$. La solution générale est $\phi(x) = A \cos(\sqrt{\lambda}x) + B \sin(\sqrt{\lambda}x)$. La condition $\phi'(0) = 0$ donne $B = 0$, puis $\phi'(1) = -A\sqrt{\lambda} \sin(\sqrt{\lambda}) = 0$ impose $\sqrt{\lambda} = n\pi$, $n \geq 0$.

$$\boxed{\phi_n(x) = \cos(n\pi x), \quad \lambda_n = (n\pi)^2, \quad n = 0, 1, 2, \dots}$$

Le mode $n = 0$ est la constante $\phi_0 = 1$, de valeur propre $\lambda_0 = 0$: il correspond à la température moyenne, conservée par l'équation de diffusion avec flux nul aux bords.

(b) Solution exacte pour $u_0(x) = \cos(\pi x)$

La condition initiale est exactement le mode ϕ_1 . Par séparation des variables, $u(x, t) = \phi_1(x) T(t)$ avec $T'(t) = -\kappa\lambda_1 T(t)$, donc $T(t) = e^{-\kappa\pi^2 t}$.

$$\boxed{u(x, t) = \cos(\pi x) e^{-\kappa\pi^2 t}.$$

Vérification :

- EDP : $\partial_t u = -\kappa\pi^2 u$ et $\kappa\partial_{xx} u = -\kappa\pi^2 u$, donc $\partial_t u - \kappa\partial_{xx} u = 0$. ✓
- CN : $\partial_x u(0, t) = -\pi \sin(0) e^{-\kappa\pi^2 t} = 0$ ✓ ; $\partial_x u(1, t) = -\pi \sin(\pi) e^{-\kappa\pi^2 t} = 0$. ✓

(c) Temps caractéristique et valeur limite

$$\boxed{\tau = \frac{1}{\kappa\lambda_1} = \frac{1}{\kappa\pi^2}.$$

Pour $\kappa = 1$: $\tau = 1/\pi^2 \approx 0,101$ s. L'amplitude décroît d'un facteur e en temps τ .

Lorsque $t \rightarrow +\infty$, $u(x, t) \rightarrow 0$ (la température s'uniformise). La valeur limite est la **moyenne initiale** : $\int_0^1 u_0(x) dx = \int_0^1 \cos(\pi x) dx = 0$.

Erreur classique : croire que la limite est la valeur initiale en un point particulier (par exemple $u_0(1/2) = 0$). C'est la *moyenne spatiale* qui est conservée, pas la valeur en un point.

2 Le jeu des erreurs : corrigé

Le code présente exactement **7 erreurs**.

- ① `np.linspace(0.0, 1.0, N)` crée N nœuds, alors qu'il en faut $N+1$ pour N éléments.
Correction : `np.linspace(0.0, 1.0, N + 1)`. Cette erreur provoque un `IndexError` lors du premier accès à `x[e+1]` pour le dernier élément.
- ② `Jac = h` est incorrect. Le jacobien de la transformation $\xi \in [-1, 1] \mapsto x \in [x_e, x_{e+1}]$ est $dx/d\xi = h/2$.
Correction : `Jac = h / 2.0`. L'erreur affecte silencieusement les intégrations : M et K sont toutes deux calculées avec un jacobien erroné.
- ③ Les fonctions de forme sont **inversées** : `Nq[0] = 0.5*(1.0+xi)` et `Nq[1] = 0.5*(1.0-xi)` sont échangées. La fonction associée au nœud gauche ($\xi = -1$) doit valoir 1 en $\xi = -1$, ce qui est $(1 - \xi)/2$.
Correction : `Nq[0] = 0.5*(1.0 - xi)`, `Nq[1] = 0.5*(1.0 + xi)`.
- ④ `dN_dx[i] = dN_dxi[i] / dxi_dx` est incorrect. Par la règle de dérivation composée : $dN/dx = (dN/d\xi)(d\xi/dx)$, soit une **multiplication** par `dxi_dx` et non une division.
Correction : `dN_dx[i] = dN_dxi[i] * dxi_dx`.
- ⑤ `J = dof[i]` doit être `J = dof[j]` : l'indice global de **colonne** de la matrice doit correspondre à l'indice j de la boucle, pas i . Cette erreur ne remplit que les termes diagonaux de la matrice locale.
Correction : `J = dof[j]`.
- ⑥ Le **poids de quadrature** `w` est absent de l'assemblage de K : `K[I, J] += kappa * dN_dx[i] * dN_dx[j] * Jac` devrait inclure `w`.
Correction : `K[I, J] += kappa * dN_dx[i] * dN_dx[j] * w * Jac`.
- ⑦ `return K, M` retourne les matrices dans le **mauvais ordre**. La convention (documentée dans l'en-tête de la fonction) est de retourner d'abord la matrice de masse, puis la matrice de rigidité.
Correction : `return M, K`.

Code corrigé :

```

1 import numpy as np
2
3 def assemble_heat_1d(N, kappa):
4     x = np.linspace(0.0, 1.0, N + 1)    # 1 corrige
5
6     M = np.zeros((N + 1, N + 1))
7     K = np.zeros((N + 1, N + 1))
8
9     xi_q = [-1.0 / np.sqrt(3.0), 1.0 / np.sqrt(3.0)]
10    w_q   = [1.0, 1.0]
11    dN_dxi = [-0.5, 0.5]
12
13    for e in range(N):

```

```
14     x0 = x[e]
15     x1 = x[e + 1]
16     h  = x1 - x0
17     Jac    = h / 2.0                # 2 corrige
18     dxi_dx = 1.0 / Jac
19
20     dof = [e, e + 1]
21
22     for q in range(2):
23         xi = xi_q[q]
24         w  = w_q[q]
25
26         Nq = [0.0, 0.0]
27         Nq[0] = 0.5 * (1.0 - xi)    # 3 corrige
28         Nq[1] = 0.5 * (1.0 + xi)    # 3 corrige
29
30         dN_dx = [0.0, 0.0]
31         for i in range(2):
32             dN_dx[i] = dN_dxi[i] * dxi_dx # 4 corrige
33
34         for i in range(2):
35             I = dof[i]
36             for j in range(2):
37                 J = dof[j]                # 5 corrige
38
39                 M[I, J] += Nq[i] * Nq[j] * w * Jac
40                 K[I, J] += kappa * dN_dx[i] * dN_dx[j] * w *
41                     Jac # 6 corrige
42     return M, K                        # 7 corrige
```